DEV-EVOLUTION-GAME

QUESTÕES DE MÉTRICAS DE PROCESSO

ASSUNTO 1: Introdução e Visão Holística das Métricas de Processo

Questão 1 — Conceitual (1 Ponto)
Pergunta: Qual é a diferença fundamental entre o foco de uma Métrica de Processo de Software e uma Métrica de Produto?
[ ] A) Métricas de processo medem o desempenho físico do servidor e banco de dados; métricas de produto medem as vendas do software.
[x] B) Métricas de processo avaliam a dinâmica, o fluxo de trabalho e o método de desenvolvimento, enquanto métricas de produto analisam a estrutura interna do código. (CORRETA)
[ ] C) Métricas de produto são calculadas apenas por gestores de negócios, enquanto métricas de processo dependem de ferramentas manuais do desenvolvedor.
[ ] D) Não existe diferença prática, pois ambas medem estritamente a quantidade de linhas físicas de código escritas na IDE.
Explicação (Flashcard): As métricas de produto medem a estrutura estática do código gerado (como tamanho e complexidade). Já as métricas de processo avaliam a dinâmica e a evolução do desenvolvimento (o fluxo de trabalho), permitindo entender a eficiência e a qualidade do time.

Questão 2 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Por que a engenharia de software prega que nenhuma métrica de processo de desenvolvimento deve ser analisada de forma isolada?
[ ] A) Porque analisar métricas de forma integrada exige computadores com alta capacidade de processamento.
[x] B) Porque métricas isoladas podem mascarar gargalos; por exemplo, uma alta produtividade pode esconder uma péssima densidade de defeitos. (CORRETA)
[ ] C) porque o uso de métricas combinadas zera automaticamente a taxa de retrabalho de requisitos.
[ ] D) Porque as métricas isoladas são incapazes de contar o número de histórias entregues em uma sprint.
Explicação (Flashcard): As métricas de processo são interdependentes. Focar em apenas uma (como produtividade) pode gerar uma ”velocidade falsa”, onde o time entrega muitas tarefas rapidamente, mas com baixa qualidade, gerando um efeito dominó de bugs e retrabalho subsequente.

Questão 3 — Exemplo Prático de Código (3 Pontos)
Pergunta: Um time comemora a entrega de 40 histórias em uma sprint (alta produtividade). Na sprint seguinte, a equipe gasta 80% do seu tempo corrigindo bugs gerados por essas mesmas histórias. Como uma visão holística classifica esse cenário?
[ ] A) Produtividade real e sustentável, pois o time manteve a entrega de novos códigos.
[ ] B) Retrabalho saudável, pois correções de bugs não interferem no ritmo da equipe.
[x] C) Velocidade falsa, onde o ganho inicial de produtividade foi anulado pela queda drástica de qualidade e pelo gargalo de retrabalho. (CORRETA)

[ ] D) Sucesso de engenharia de software, já que os testes em produção foram automatizados de forma transparente.
Explicação (Flashcard): O aumento de bugs gera mais refação imediata, travando o sistema. A alta produtividade isolada escondeu a queda da qualidade (alta densidade de defeitos), e o excesso de retrabalho estancou a equipe na sprint seguinte, reduzindo a produtividade efetiva.

Questão 4 — DESAFIO (3 Pontos)
Pergunta: Um gerente de projetos nota que um módulo específico obteve ”densidade de defeitos igual a zero” durante a fase de testes. Contudo, a taxa de retrabalho de codificação da equipe foi de 45% na mesma sprint. O que essa análise combinada de processo revela?
[x] A) Que a baixa densidade de defeitos é falsa; os bugs e falhas de requisitos estavam sendo corrigidos exaustivamente via retrabalho de codificação antes de chegarem à fase formal de testes. (CORRETA)
[ ] B) Que a equipe atingiu a maturidade máxima de processo, zerando os bugs e agilizando as entregas finais.
[ ] C) Que a taxa de retrabalho alta foi causada unicamente pela inexperiência da equipe de testes.
[ ] D) Que a complexidade ciclomática das classes do sistema impede a identificação de defeitos reais.
Explicação (Flashcard): A interdependência das métricas mostra que um dado isolado engana. Uma densidade de defeitos nula nos testes pode ser fruto de um retrabalho massivo anterior na fase de codificação para corrigir erros de requisitos mal interpretados, mascarando falhas críticas no fluxo inicial de design e comunicação.

ASSUNTO 2: Produtividade de Software

Questão 5 — Conceitual (1 Ponto)
Pergunta: Como é definida a métrica de Produtividade em engenharia de software?
[ ] A) A quantidade de horas extras trabalhadas individualmente por cada membro da equipe.
[ ] B) O número total de linhas físicas contadas no arquivo de banco de dados do sistema.
[x] C) A quantidade de software útil produzido pelo time por unidade de esforço ou tempo investido. (CORRETA)
[ ] D) A velocidade nominal de processamento e execução das instruções lógicas em produção.
Explicação (Flashcard): A produtividade é uma métrica de processo que avalia a eficiência da equipe, medindo o volume de software produzido (saída/output) por unidade de esforço empregado no projeto (entrada/input).

Questão 6 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é o principal benefício gerencial de monitorar a produtividade média da equipe e qual é o seu principal cuidado técnico?
[ ] A) Permite programar o desligamento de servidores ociosos; cuidado para não apagar o banco de dados.
[x] B) Apoia a estimativa de prazos de entregas futuras; cuidado porque alta produtividade quantitativa não garante a qualidade técnica do código. (CORRETA)
[ ] C) Substitui a necessidade de manter uma equipe de garantia de qualidade; cuidado para não deixar bugs escaparem.
[ ] D) Reduz o custo do desenvolvimento de forma linear; cuidado para não violar os requisitos do cliente.
Explicação (Flashcard): Mapear a produtividade ajuda a estimar cronogramas e capacidades futuras de sprints. Contudo, o time deve ter cuidado, pois focar apenas em produzir mais pode

comprometer a qualidade do código (gerando mais defeitos estruturais e dívida técnica).

Questão 7 — Exemplo Prático de Código (3 Pontos)
Pergunta: Com base na Atividade 12 do material de aula, um time de 5 pessoas trabalhou por 2 semanas (8h/dia, de segunda a sexta, totalizando 10 dias de trabalho) e entregou exatamente 20 histórias. Qual foi a produtividade em histórias/hora de esforço do período?
[x] A) 0.05 histórias/hora. (CORRETA)
[ ] B) 0.50 histórias/hora.
[ ] C) 4.00 histórias/hora.
[ ] D) 0.25 histórias/hora.
Explicação (Flashcard): O esforço total de entrada (Input) foi de: 5 pessoas$\times 10$ dias$\times 8$ horas/dia $=$ 400 horas. A saída (Output) foi de 20 histórias. Aplicando a fórmula: Produtividade $= \frac{20 \text{ histórias}}{400 \text{ horas}} =$ 0.05 histórias/hora.

Questão 8 — DESAFIO (3 Pontos)
Pergunta: Por que frameworks ágeis modernos preferem medir a produtividade por "Pontos de Função ou Histórias de Usuário entregues" em detrimento da contagem tradicional de "Linhas de Código (LOC) escritas"?
[ ] A) Porque contar linhas de código exige que a equipe realize a medição manualmente todos os dias.
[ ] B) Porque a métrica LOC é incompatível com sistemas desenvolvidos em nuvem pública ou privada.
[x] C) Porque o LOC incentiva códigos redundantes e verbosos, enquanto histórias de usuário medem o real valor funcional entregue ao cliente. (CORRETA)
[ ] D) Porque a contagem de pontos de função é imune a quaisquer variações de requisitos ou escopo.
Explicação (Flashcard): Medir produtividade usando LOC gera o incentivo perverso de escrever código inflado e complexo. Ao usar histórias de usuário ou pontos de função, mede-se a produtividade com base no valor de negócio real entregue ao cliente, promovendo códigos enxutos, fáceis de manter e de alta qualidade.

ASSUNTO 3: Densidade de Defeitos e Taxa de Escape

Questão 9 — Conceitual (1 Ponto)
Pergunta: O que a métrica de Densidade de Defeitos mede e qual a importância de sua normalização?
[x] A) Mede a quantidade de bugs identificados de forma normalizada pelo tamanho do software, permitindo comparar a qualidade de módulos de tamanhos diferentes. (CORRETA)
[ ] B) Mede o tempo médio gasto pelos desenvolvedores para codificar cada teste unitário em homologação.
[ ] C) Conta a quantidade de commits de refatoração enviados para o repositório principal no mês.
[ ] D) Avalia o número de falhas de hardware que paralisaram a execução do sistema em produção.
Explicação (Flashcard): A densidade de defeitos quantifica os bugs encontrados normalizando-os pelo tamanho do software (KLOC ou FP). Essa divisão é vital para comparar de forma justa a estabilidade e qualidade de módulos pequenos e grandes.

Questão 10 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Se um módulo do sistema apresenta uma densidade de defeitos extremamente baixa (próxima a zero), como a equipe de QA deve interpretar este resultado?

[ ] A) O código é 100% perfeito e está livre de falhas de lógica estrutural.
[ ] B) Os requisitos do cliente foram mapeados sem nenhum desvio ou mal-entendido.
[x] C) Deve-se ter cautela; o resultado pode indicar alta estabilidade ou que a estratégia de testes foi ineficiente e não achou os bugs. (CORRETA)
[ ] D) O módulo não precisa de testes de regressão antes de subir para produção.
Explicação (Flashcard): Uma baixa densidade de defeitos é excelente se o código estiver realmente estável. Contudo, ela pode mascarar testes ruins, onde os analistas de QA aplicaram cenários fracos que simplesmente falharam em expor e catalogar as vulnerabilidades e bugs reais do código.

Questão 11 — Exemplo Prático de Código (3 Pontos)
Pergunta: Com base nos dados da Atividade 12, foram identificados e documentados exatamente 14 defeitos em um módulo estimado em 70 Pontos de Função (FP). Qual é a densidade de defeitos do módulo?
[ ] A) 5.00 defeitos/FP.
[ ] B) 0.50 defeitos/FP.
[x] C) 0.20 defeitos/FP. (CORRETA)
[ ] D) 1.40 defeitos/FP.
Explicação (Flashcard): Aplicando a fórmula base: Densidade de Defeitos $= \frac{\text{Nº de Defeitos}}{\text{Tamanho (FP)}}$. Logo, dividimos 14 defeitos por 70 FP, resultando em exatamente 0.20 defeitos/FP.

Questão 11 — DESAFIO (3 Pontos)
Pergunta: Um sistema obteve densidade de defeitos baixíssima em homologação, mas ao ser implantado em produção, apresentou dezenas de bugs reportados pelos usuários. Qual indicador expõe essa falha e o que ele reflete?
[ ] A) Baixo Churn de código, refletindo estabilidade nas classes do sistema.
[ ] B) Alto DIT, indicando heranças muito profundas que confundiram o usuário final.
[x] C) Alta Taxa de Escape, refletindo que os testes internos foram feitos em cenários distantes da realidade do usuário. (CORRETA)
[ ] D) WMC elevado, indicando que as classes possuem métodos redundantes em produção.
Explicação (Flashcard): A Taxa de Escape mede os bugs que vazaram dos testes internos e chegaram à produção. Um escape alto sinaliza um gargalo grave no processo de testes: os ambientes de QA podem ser irreais, os dados de testes insuficientes ou os cenários de uso divergentes das ações dos clientes reais.

ASSUNTO 4: Taxas de Retrabalho (Rework Rates)

Questão 12 — Conceitual (1 Ponto)
Pergunta: Como é definido o conceito de ”Retrabalho” (Rework) no fluxo de processo de software?
[ ] A) O esforço de programar novos recursos solicitados de última hora pelo cliente.
[ ] B) A quantidade de reuniões diárias exigidas para alinhar o cronograma da sprint.
[x] C) A quantidade de esforço ou tempo gasto para refazer, corrigir ou alterar tarefas após a sua entrega inicial. (CORRETA)
[ ] D) O tempo gasto instalando dependências e configurando a IDE local dos desenvolvedores.
Explicação (Flashcard): O retrabalho é a métrica do desperdício de tempo e recursos do projeto. Ele representa as horas jogadas fora para corrigir erros ou reinterpretar requisitos mal definidos após uma entrega que deveria estar finalizada.

Questão 13 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Qual é o impacto direto de uma alta taxa de retrabalho constante na capacidade

produtiva da equipe de desenvolvimento?
[x] A) Reduz drasticamente a produtividade efetiva e drena o tempo que o time usaria para criar inovação e novas features. (CORRETA)
[ ] B) Melhora a legibilidade do código ao forçar refatorações constantes em produção.
[ ] C) Diminui de forma forçada a profundidade de herança (DIT) de todas as classes do núcleo.
[ ] D) Isola as classes do sistema, diminuindo o acoplamento medido pelo indicador CBO.
Explicação (Flashcard): Com taxas de retrabalho altas, a equipe fica presa apagando incêndios e refazendo tarefas mal especificadas ou cheias de bugs. O tempo gasto corrigindo código antigo impede que a equipe foque em entregar valor de negócio novo ao cliente, estancando a evolução do projeto.

Questão 14 — Exemplo Prático de Código (3 Pontos)
Pergunta: Com base na Atividade 12, em um período com esforço total disponível de 400 horas, o time gastou 60 horas em correções de código e 40 horas refazendo requisitos mal entendidos. Qual a taxa de retrabalho por esforço (%) desse ciclo?
[ ] A) 10%.
[ ] B) 15%.
[x] C) 25%. (CORRETA)
[ ] D) 30%.
Explicação (Flashcard): Somamos as horas de desperdício (refação): 60 horas (correções) $+$ 40 horas (requisitos) $=$ 100 horas de retrabalho. Dividimos pelo esforço total disponível (400 horas) e multiplicamos por 100: Taxa de Retrabalho $= \frac{100}{400} \times 100 = 25\%$.

Questão 15 — DESAFIO (3 Pontos)
Pergunta: Uma equipe ágil possui uma taxa de retrabalho persistente de 35% causada principalmente por "requisitos mal interpretados". Qual prática de processo ataca a causa raiz desse gargalo de forma definitiva?
[ ] A) Dobrar a quantidade de linhas de código escritas em testes de estresse em homologação.
[ ] B) Penalizar os desenvolvedores que não conseguirem decifrar as especificações do cliente.
[x] C) Adotar técnicas colaborativas como BDD (Behavior-Driven Development) e reuniões de refinamento (grooming) com o cliente antes de codificar. (CORRETA)
[ ] D) Migrar toda a base de código orientada a objetos para o paradigma procedimental simples.
Explicação (Flashcard): O retrabalho por falha de requisitos ocorre por desalinhamento inicial de comunicação. Práticas colaborativas (como refinamentos de escopo frequentes e a escrita de cenários BDD claros junto ao cliente/PO) garantem que o time codifique exatamente o esperado, eliminando a refação pós-entrega.

ASSUNTO 5: O Custo do Retrabalho no Tempo (Custo Exponencial)

Questão 16 — Conceitual (1 Ponto)
Pergunta: O que dita a lei do custo exponencial do retrabalho em engenharia de software?
[x] A) Quanto mais tarde um defeito é encontrado no ciclo de desenvolvimento, exponencialmente mais caro é para consertá-lo. (CORRETA)
[ ] B) O preço das ferramentas de análise estática aumenta de acordo com as linhas de código (LOC).
[ ] C) O custo financeiro de refação de bugs em produção diminui linearmente com o tempo de projeto.
[ ] D) O tempo gasto para reescrever um documento de requisitos é idêntico ao tempo de codificação local.

Explicação (Flashcard): Encontrar e corrigir falhas nas fases iniciais do projeto (como especificação de requisitos) custa uma fração insignificante em relação ao custo astronômico e aos danos de se corrigir o mesmo bug após o software estar rodando em produção.

Questão 17 — Vantagens e Desvantagens (2 Pontos)
Pergunta: Sob a ótica do custo exponencial do retrabalho, qual é o principal objetivo de adotarmos práticas conhecidas como "Shift-Left"?
[ ] A) Mover toda a infraestrutura física de servidores locais para ambientes em nuvem.
[x] B) Antecipar as etapas de testes e validações para as fases iniciais do desenvolvimento, economizando tempo e recursos. (CORRETA)
[ ] C) Substituir a equipe de desenvolvimento por analistas de suporte em produção.
[ ] D) Reduzir o número total de requisitos funcionais que precisam de testes unitários.
Explicação (Flashcard): O "Shift-Left" prega antecipar os testes. Ao validar requisitos e lógicas de código logo no início do ciclo (Requirements/Codificação), interceptamos os defeitos antes que eles se propaguem para as fases caras de QA (100x) ou de Produção (1000x), maximizando o orçamento do projeto.

Questão 18 — Exemplo Prático de Código (3 Pontos)
Pergunta: Um erro crítico no cálculo de faturamento foi pego na fase de Requisitos (Custo 1x). O mesmo bug passou despercebido e vazou para Produção (Custo 1000x). Qual fator justifica esse salto exponencial de custo na última fase?
[ ] A) O preço fixo de faturamento exigido pelos frameworks de bancos de dados modernos.
[ ] B) A necessidade de reescrever apenas a documentação textual do sistema em homologação.
[x] C) Hotfixes emergenciais rápidos, parada de faturamento, cancelamento de notas incorretas, multas fiscais e danos graves à marca. (CORRETA)
[ ] D) O esforço simples de recodificar o arquivo localmente usando a IDE do programador sênior.
Explicação (Flashcard): Corrigir em produção (1000x) envolve um cenário caótico: exige mobilizar equipe extra sob estresse, gera retrabalho legal (cancelar e reemitir notas fiscais erradas), expõe a empresa a multas fiscais pesadas, irrita os clientes reais e causa prejuízos diretos à credibilidade e imagem da organização no mercado.

Questão 19 — DESAFIO (3 Pontos)
Pergunta: Uma diretoria financeira resiste à implementação de testes unitários automáticos na codificação alegando que "desenvolvedores perdem tempo demais escrevendo testes em vez de criarem novas features". Como a curva de custo exponencial rebate este argumento?
[ ] A) Provando que testes em produção (1000x) são mais baratos por serem feitos por usuários reais.
[ ] B) Mostrando que a complexidade de resposta (RFC) das classes zera quando testes são omitidos.
[x] C) Provando que achar bugs na codificação (10x) é 10 vezes mais barato que em QA (100x) e 100 vezes mais barato que em produção (1000x). (CORRETA)
[ ] D) Provando que o tempo de codificação de testes unitários é processado sem consumo de esforço.
Explicação (Flashcard): O tempo gasto pelos desenvolvedores escrevendo testes unitários preventivos na fase de codificação é um investimento de altíssimo retorno. Interceptar e matar os bugs localmente (Custo 10x) evita o desperdício de tempo em cascata de QA (Custo 100x) e impede catástrofes em produção (Custo 1000x), economizando milhares de horas e recursos no médio e longo prazo.